



M2 IoT

sujet 3

**Évaluation IoT – Projet Intégré RFID,
ESP32 et Machine Learning**

Auteur : Malik HARRIZ

1 Évaluation IoT – Projet Intégré RFID, ESP32 et Machine Learning

1.1 □ Contexte

Dans le cadre de cette évaluation, vous devrez concevoir et développer un **système IoT complet** intégrant :

- un **capteur RFID** pour l'identification
- une carte **ESP32** pour la collecte et transmission des données
- une **base de données**
- une application en **Python**
- un module de **machine learning** exploitant les données collectées

1.2 Objectifs

- Concevoir une architecture IoT complète (hardware + software)
- Manipuler un capteur RFID avec ESP32
- Implémenter une communication entre un objet connecté et un serveur
- Structurer et exploiter une base de données
- Développer une application Python
- Appliquer des techniques de machine learning sur des données réelles

Vous devez développer un système permettant de :

1. **Identifier des utilisateurs via RFID**
2. **Enregistrer leurs passages dans une base de données**
3. **Analyser les comportements d'utilisation via un modèle de machine learning**

1.3 Spécifications techniques

1.4 Partie embarquée (ESP32 + RFID)

- Lire les identifiants RFID (UID)
- Associer chaque carte à un utilisateur
- Envoyer les données vers un serveur via Wi-Fi
- Format recommandé : JSON
- Gérer les erreurs (lecture RFID, réseau)

1.5 Backend et base de données

- Concevoir une base de données (SQL ou NoSQL)
- Stocker les données suivantes :
 - UID RFID
 - Timestamp
 - Événement (entrée / sortie / autorisation)
 - Localisation (optionnel)
- Implémenter un mécanisme de réception :
 - API REST ou MQTT recommandé

1.6 Base de données - MCD

1.7 Entités principales

- Mise en place d'une base de données (MySQL, PostgreSQL, MongoDB...)

1.7.1 UTILISATEUR

- id_utilisateur (PK)
- nom
- prénom
- email
- type_utilisateur (admin, visiteur, employé...)

1.7.2 BADGE_RFID

- id_badge (PK)
- uid (unique RFID)
- date_attribution
- statut (actif, inactif)

1.7.3 LECTEUR (ESP32)

- id_lecteur (PK)
- nom
- localisation
- adresse_ip
- statut

1.7.4 ÉVÉNEMENT

- id_evenement (PK)
- timestamp
- type_evenement (entrée, sortie, refus)
- resultat (autorisé, refusé)
- id_badge (FK)

- id_lecteur (FK)

1.7.5 DONNEE_ML

- id_donnee_ml (PK)
- id_evenement (FK)
- feature_1 (ex: heure)
- feature_2 (ex: fréquence d'accès)
- feature_3 (ex: durée présence simulée)
- label (optionnel)
- prediction (optionnel)

1.8 Base de données - Relations

1.8.1 UTILISATEUR — possède — BADGE_RFID

- Un utilisateur peut avoir **1 ou plusieurs badges**
- Un badge appartient à **1 seul utilisateur**

→ Relation : 1,N

1.8.2 BADGE_RFID — génère — ÉVÉNEMENT

- Un badge peut générer **plusieurs événements**
- Un événement est lié à **un seul badge**

→ Relation : 1,N

1.8.3 LECTEUR — enregistre — ÉVÉNEMENT

- Un lecteur ESP32 enregistre **plusieurs événements**
- Un événement est capturé par **un seul lecteur**

→ Relation : 1,N

1.8.4 ÉVÉNEMENT — alimente — DONNEE_ML

- Un événement peut produire **0 ou 1 donnée ML**
- Une donnée ML est basée sur **un seul événement**

→ Relation : 1,1 (optionnelle côté événement)

1.9 Base de données - Vue simplifiée (notation textuelle)

```
UTILISATEUR (1,N) — possède — (1,1) BADGE_RFID
BADGE_RFID (1,N) — génère — (1,1) ÉVÉNEMENT
LECTEUR (1,N) — enregistre — (1,1) ÉVÉNEMENT
ÉVÉNEMENT (1,1) — alimente — (0,1) DONNEE_ML
```

1.10 Extensions possibles (bonus)

1.10.1 LOCALISATION (optionnel)

- id_localisation (PK)
- nom
- bâtiment
- étage

→ relation avec LECTEUR

1.10.2 AUTORISATION

- id_autorisation (PK)
- id_utilisateur (FK)
- zone
- horaires_autorisés

1.11 Application Python

L'application Python doit permettre de :

- Recevoir et traiter les données
- Interagir avec la base de données
- Visualiser les données (logs, graphiques simples)

1.12 Machine Learning

À partir des données collectées :

- Construire un dataset exploitable
- Implémenter un modèle :
 - Détection d'anomalies
 - Clustering d'utilisateurs
 - Prédiction de fréquentation

- Justifier :
 - le choix du modèle
 - les variables utilisées
 - les résultats obtenus

1.13 Livrables attendus

1.13.1 Code source

- Code ESP32
- Code Python
- Scripts base de données

1.13.2 Présentation orale

- Architecture globale
 - Schémas du système
 - Choix techniques
 - Partie machine learning
-

1.13.3 Démonstration

- Lecture RFID fonctionnelle
- Transmission des données
- Stockage en base
- Analyse via modèle ML

1.14 Planning suggéré (5 jours)

- **Jour 1** : conception et architecture
- **Jour 2** : ESP32 + RFID
- **Jour 3** : backend + base de données
- **Jour 4** : Python + machine learning
- **Jour 5** : intégration + maintenance

1.15 Critères d'évaluation

Critère	Pondération
Fonctionnement RFID	20%
Qualité du code	20%

Critère	Pondération
Base de données	15%
Machine Learning	25%
Architecture globale	10%
Documentation & présentation	10%

1.16 Bonus

- Dashboard (Streamlit, Flask...)
- Sécurité (authentification, chiffrement)
- Déploiement cloud
- Multi-capteurs / multi-ESP32

1.17 Contraintes

- Utilisation obligatoire de **Python**
- Utilisation obligatoire d'une **base de données**
- Utilisation obligatoire d'un **capteur RFID**
- Le machine learning doit exploiter **les données collectées**

1.18 □ Suggestions techniques

- Communication : HTTP, MQTT
- Base de données : SQLite, PostgreSQL, MongoDB
- Python : Flask, FastAPI, Pandas, Scikit-learn
- Visualisation : Matplotlib, Seaborn